

Build Your Own AI Job Hunter

A complete Claude Code + local dashboard build guide

A step-by-step Karmendra AI manual for building a local system that scans public company job boards, scores roles against your profile, tailors truthful resumes, prepares applications, and shows the workflow in a live dashboard.

Built for people who want a practical project they can run on their own computer - without needing to write the application by hand.

WHAT YOU WILL BUILD

- A Node.js job-hunting engine that reads public ATS feeds.
- A Next.js dashboard at <http://localhost:4319>.
- A shared local data layer using JSON and JSONL files.
- A scanner for Greenhouse and Lever, with extension points for other ATS providers.
- An A-F scoring engine and match percentage.
- A truthful per-job resume tailoring workflow.
- A Playwright browser workflow that fills clean application forms and parks anything uncertain.
- A human-review queue for CAPTCHAs, legal questions, account walls, and unknown fields.
- Live dashboard updates, command buttons, dry-run mode, and a repeatable hunt cycle.

IMPORTANT Start in dry-run mode. Do not bypass CAPTCHAs, do not fabricate experience, and do not let the system guess work authorization, sponsorship, salary, EEO, or other legal answers.

Karmendra AI | Instagram: @Karmendra.ai

1. How the system is structured

The build has two independent parts. The engine does the work. The dashboard only reads the same local files and sends commands into a queue. This keeps the system easy to inspect and easy to recover if something breaks.

ENGINE	SHARED DATA	DASHBOARD
Node.js	JSON / JSONL	Next.js

Data flow: public job-board feeds -> scanner -> score -> tailored CV -> apply / park -> shared files -> live dashboard.

Folder layout

```
karmendra-job-hunter/  
  package.json  
  engine/  
    providers/  
  data/  
    config/  
    state/  
    log/  
    queue/  
    cv/  
  web/  
  .claude/  
    commands/
```

Core local files

- data/config/profile.json - your identity, targets, compensation preferences, and truthful answers.
- data/config/prefs.json - autonomy, dry/live apply, score thresholds, caps, and scan interval.
- data/config/companies.json - enabled company boards and ATS tokens.
- data/state/jobs.json - all discovered roles and their current pipeline state.
- data/state/runs.json - history of hunt cycles.
- data/log/activity.jsonl - append-only live activity feed.
- data/queue/commands.jsonl - commands sent from dashboard to engine.
- data/cv/base.md - your original CV; per-job variants are written beside it.

2. Prerequisites and one-time setup

Install these before opening Claude Code:

- Claude Desktop or Claude Code CLI with a plan that includes Claude Code.
- Node.js LTS.
- Git.
- A modern Chromium-based browser (Playwright installs its own browser runtime too).

BILLING NOTE If you want Claude Code to use your subscription session rather than an API key, leave `ANTHROPIC_API_KEY` unset. If you already use API billing intentionally, keep your existing setup and understand that usage can be billed separately.

Create an empty project folder

```
mkdir karmendra-job-hunter
cd karmendra-job-hunter
```

On Windows you can create the folder in File Explorer and open a terminal inside it instead.

Start Claude Code

```
claude --dangerously-skip-permissions --model best
```

CAUTION `--dangerously-skip-permissions` allows Claude Code to make changes without asking for each permission. Use it only inside a new project folder you control. If you prefer confirmation prompts, start Claude normally.

Inside Claude Code, increase reasoning effort if your client exposes the `/effort` command:

```
/effort
```

Choose the highest effort level available to you for the initial build. Then paste the prompts in this guide in order. Let Claude finish and fix errors before moving to the next step.

Rule for the whole build

1. Paste one prompt.
2. Let Claude create or edit the files.
3. Ask it to run the checks named in the prompt.
4. Do not continue while there are unresolved syntax, dependency, or type errors.
5. Only then paste the next prompt.

3. Step 1 - Scaffold the engine and dashboard

This creates the two-part app and locks the main dependencies. The dashboard and engine should not call each other directly; they communicate through files under data/.

PASTE INTO CLAUDE CODE - PROMPT 1

You are my build partner for a local project called Karmendra Job Hunter. Work patiently and explain what you changed after each major action. Do not delete unrelated files and do not start any paid service.

Build a two-part local application in this empty folder:

1. A Node.js engine in engine/ that scans jobs, scores them, tailors CVs, and prepares or submits applications.
2. A Next.js dashboard in web/ that runs at `http://localhost:4319`.
3. The engine and dashboard must communicate only through files under data/. Do not create an internal network API between them.

Root engine requirements:

- package.json name: karmendra-job-hunter-engine
- private: true
- type: module
- scripts: cycle = node engine/cycle.mjs --force; apply = node engine/apply.mjs; seed = node engine/seed.mjs
- dependencies: marked ^14.1.4 and playwright 1.58.1

Create these folders: engine/, engine/providers/, data/config/, data/state/, data/log/, data/queue/, data/cv/.

Create web/ as a Next.js App Router + TypeScript application. Pin Next 16.2.9, React 19.2.4, and react-dom 19.2.4. Add @base-ui/react ^1.5.0, class-variance-authority ^0.7.1, clsx ^2.1.1, lucide-react ^1.17.0, motion ^12.40.0, next-themes ^0.4.6, react-markdown ^10.1.0, shadcn ^4.11.0, sonner ^2.0.7, tailwind-merge ^3.6.0, and tw-animate-css ^1.4.0. Use Tailwind 4 with @tailwindcss/postcss 4, TypeScript 5, ESLint 9, eslint-config-next 16.2.9, and React 19 type packages.

Set web scripts so dev and start use port 4319. Add web/lib/Utils.ts with a standard cn() helper built from clsx + tailwind-merge. Before writing application code later, inspect the version-specific docs installed under web/node_modules/next/dist/docs/ rather than assuming older Next.js behavior.

Install the root dependencies and Playwright browser runtime, then install the dashboard dependencies. Do not build features yet. At the end, print the folder tree and tell me exactly how to verify that the blank dashboard starts on port 4319.

Terminal commands Claude should ultimately run

```
npm install
npx playwright install
npm --prefix web install
```

Expected result: root package.json, engine/, data/, and web/ exist; the blank Next.js app can start on port 4319.

4. Step 2 - Build the shared data contract

The engine and dashboard need to agree on exactly what a job, profile, preference set, activity event, and command look like.

PASTE INTO CLAUDE CODE - PROMPT 2

Continue Karmendra Job Hunter. Build a shared flat-file data layer for the dashboard. Create `web/lib/types.ts`, `web/lib/paths.ts`, and `web/lib/data.ts`.

In `types.ts` define:

- ATS values: greenhouse, lever, ashby, workable, workday, other.
- Grades A through F.
- Job statuses: new, scored, cv_ready, needs_you, applied, responded, interview, offer, rejected, discarded, skip.
- Park reasons: captcha, email_verification, account_wall, workday, legal_question, ghost, low_fit, unknown_form.
- Types/interfaces for Salary, Job, EeoAnswers, LegalAnswers, Profile, Prefs, Company, ActivityEvent, Run, Command, and Snapshot.

A Job must include a stable id, company, title, URL, location, ATS, timestamps, status, optional salary, score, grade, match percentage, archetype, rationale, reasons, gaps, legitimacy, CV paths, application time, and park information.

Profile must hold contact details, links, headline, target roles, optional archetypes/superpowers/deal-breakers, compensation preferences, legal answers, onboarded, isDemo, and updatedAt. Prefs must support autonomy, autoSubmit, liveApply, score threshold, daily cap, per-company cap, scan interval, working hours, theme, and blocked companies.

In `paths.ts` resolve the shared data directory from `FABJOB_DATA_DIR` when provided, otherwise from `../data` relative to `web/`. Export paths for config, state, log, queue, CV, profile, prefs, companies, jobs, runs, activity, and commands.

In `data.ts` implement safe helpers to read JSON with fallbacks, read JSONL, atomically write JSON, append JSONL, load a complete Snapshot, save profile/prefs/companies, queue commands, and produce a stat signature based on file modification time and size. Use sensible defaults: autonomy false, auto-submit enabled but liveApply false, minimum score 4.0, daily cap 12, per-company cap 2, scan every 30 minutes, working hours 9-19.

Run TypeScript/lint checks that are available. Do not add business logic yet. Explain the data files that now connect the engine and UI.

Expected result: the web app has a typed, safe local data layer and still runs without type errors.

5. Step 3 - Build the public ATS scanner

The scanner should use public job-board feeds. Start with Greenhouse and Lever. Keep Ashby and Workable as provider stubs until you intentionally implement their public-feed format.

PASTE INTO CLAUDE CODE - PROMPT 3

Continue Karmendra Job Hunter. Build the engine scanner and provider system.

Create engine/lib.mjs with shared paths into data/, JSON read/write helpers, atomic writes, appendActivity(), getJobs(), saveJobs(), addRun(), and updateJob(). Make writes resilient to transient rename/lock errors.

Create engine/providers/_http.mjs with a small fetch wrapper that:

- uses AbortController with a timeout around 9 seconds,
- sends a clear local-tool user agent,
- returns parsed JSON for successful responses and null on failures.

Create Greenhouse and Lever providers with the same interface: async fetch(entry, ctx) -> array of {title, url, location, salary, postedAt}.

Greenhouse endpoint: <https://boards-api.greenhouse.io/v1/boards/<token>/jobs>

Lever endpoint: <https://api.lever.co/v0/postings/<token>?mode=json>

Create ashby.mjs and workable.mjs with the same interface but initially return an empty array and leave a clear TODO comment to add a verified public feed later.

Create engine/scan.mjs. It should load enabled companies and the user's target roles, build positive keywords from the role list plus useful defaults such as AI, ML, LLM, agent, engineer, developer, product, data, platform, solutions, automation, architect, analyst, manager, scientist, research, growth, marketing, content, operations, strategy, and designer. Exclude internships/co-op/apprentice titles.

For each enabled company, call the provider, keep a limited number of relevant roles per company, create a stable dedupe id from ATS + board token + the last URL segment, skip jobs already in jobs.json, and prepend fresh jobs with status new and timestamps. Add a short delay between boards. Append a scan activity event with board count and number of new roles. Export scan() and allow direct execution with node engine/scan.mjs.

Do not run the scanner yet if no company list exists. Show me the created provider files and explain how I add a Greenhouse or Lever company token later.

Expected result: engine/lib.mjs, providers, and engine/scan.mjs exist and can discover jobs once companies.json has enabled boards.

6. Step 4 - Build the A-F scoring engine

The first scorer can be deterministic and local. Later, the /hunt-cycle Claude command will use actual CV-vs-job judgment to refine strong matches.

PASTE INTO CLAUDE CODE - PROMPT 4

Continue Karmendra Job Hunter. Create engine/score.mjs. It must score jobs whose status is new against the user's target roles and profile, then save grade, numeric score, match percentage, archetype, reasons, gaps, rationale, legitimacy, and status scored.

Use a 1.0-5.0 internal score. Grade thresholds:

A: >= 4.5

B: >= 4.0

C: >= 3.3

D: >= 2.6

E: >= 1.8

F: below 1.8

The heuristic should reward a strong target-role phrase match, multiple useful title-token matches, seniority when relevant, remote/hybrid/global location, and salary presence. Add a small stable hash-based variation from the job id so repeated runs are deterministic.

Map titles into practical archetypes such as Agentic/Automation, AI Forward Deployed, AI Solutions Architect, Technical AI PM, AI Platform/LLMOps, and a fallback AI/Engineering category.

Generate plain-English reasons from real signals only. Generate gaps/watch-outs only when supported. Convert the numeric score into a human match percentage. Mark obvious job-board records as high legitimacy unless there is a specific reason to be cautious.

Process all new jobs, save the updated list once, append an activity summary, and also append a highlight event for the strongest A-grade role if one exists. Export score() and support direct execution.

After saving the file, explain the grade thresholds in one sentence and run a syntax check.

Expected result: newly discovered jobs move from new -> scored and gain A-F grades plus match percentages.

7. Step 5 - Tailor a truthful CV for strong matches

This stage should never invent achievements or experience. It can reorder, summarize, and emphasize only facts already present in the base CV.

PASTE INTO CLAUDE CODE - PROMPT 5

Continue Karmendra Job Hunter. Create engine/tailor.mjs.

Requirements:

- Read data/cv/base.md.
- Only target jobs with grade A or B, status scored, and no existing tailored CV.
- Write one Markdown CV per job at data/cv/<jobId>.md.
- Preserve the actual Experience section and factual claims from the base CV. Do not add employers, dates, projects, skills, metrics, degrees, or achievements that are not present.
- Create a role-specific Summary that reframes the candidate's existing strengths for the job. You may choose a focus based on the title (software/AI, operations, media/growth, engineering, etc.).
- Add a short Why <Company> section using the existing score reasons rather than invented company research.
- Add a small footer noting the target role, match percentage, and grade.
- Update the job to status cv_ready with cvPath and pdfPath fields.
- Append an activity event for each tailored CV.
- Export tailorAuto(limit = 8) and support direct execution.

If the base CV is missing, fail with a clear message telling the user to paste a CV first. At the end, show me how to create data/cv/base.md manually for testing.

TRUTH RULE The CV tailor may change emphasis and wording, but it must not fabricate facts. Treat this as a hard requirement, not a style preference.

8. Step 6 - Build the browser apply engine

This is the most sensitive component. Build it so dry-run is the default, uncertain fields are parked, and CAPTCHAs are never bypassed.

PASTE INTO CLAUDE CODE - PROMPT 6

Continue Karmendra Job Hunter. Build the application engine in engine/apply.mjs and a PDF helper in engine/render-pdf.mjs. Use Playwright Chromium.

render-pdf.mjs:

- export async function renderPdf(name).
- read data/cv/<name>.md, render Markdown to basic print-friendly HTML with marked, and use Playwright to print data/cv/<name>.pdf.

apply.mjs safety requirements:

1. Dry-run is the default. A dry run may open and fill forms but must never press the final submit button.
2. Never solve or bypass CAPTCHAs. If detected, save a screenshot and park the job as needs_you with reason captcha.
3. Never invent work authorization, sponsorship, salary, start date, criminal disclosure, security clearance, EEO, or other legal/identity answers. Use only values stored in profile. If a required question cannot be answered safely, park it.
4. Never apply when profile.isDemo is true.
5. Respect minimum score, daily application cap, and per-company cap.
6. Pace actions with small randomized delays.

Implementation behavior:

- Load profile, prefs, jobs, and the tailored CV. Generate the CV PDF if needed.
- Support guest application flows for Greenhouse, Lever, and Ashby where the current public form can be handled without login. Derive an apply URL when the ATS uses a separate apply path.
- Inspect visible inputs, textareas, selects, radio groups, checkboxes, and file fields. Derive a human-readable label using aria-label, associated label, wrapping label, legend, placeholder, or name.
- Map common labels to profile values: first/last/full name, email, phone, location, LinkedIn, GitHub, portfolio, current employer/title, work authorization, sponsorship, relocation, notice period, salary expectation, start date, referral source, EEO answers, and other explicitly stored truthful values.
- Upload the tailored PDF to resume/CV fields.
- Track unanswered required fields. If any remain, park rather than guessing.
- Take a screenshot before submission.
- In live mode, find a clear final submit button, click it once, wait, then confirm success from page text, URL, or the form disappearing without validation errors. If confirmation is unclear, park for verification rather than claiming success.
- On success update status applied and appliedAt. On a wall/error update status needs_you with an appropriate parkedReason and parkedNote. Append activity for every outcome.

Add CLI handling so `node engine/apply.mjs` is dry-run and `node engine/apply.mjs --live` enables real submission only when prefs/profile allow it. Run a syntax check and explain every condition that prevents a real submit.

DO NOT AUTOMATE AROUND A WALL CAPTCHA, login/account gates, email verification, or ambiguous legal questions belong in the human-review queue. The goal is reliable assistance, not bypassing a site's controls.

9. Step 7 - Wire the engine into one hunt cycle

A cycle should process dashboard commands, scan, score, tailor, log the run, and optionally start application preparation.

PASTE INTO CLAUDE CODE - PROMPT 7

Continue Karmendra Job Hunter. Build engine/commands.mjs, engine/cycle.mjs, and engine/mark.mjs.

commands.mjs:

- Read data/queue/commands.jsonl.
- Parse valid JSON lines and process only commands not yet marked done.
- Support at least discard, submit/mark applied, scan_now, pause, resume, and any preference/company commands already used by the UI.
- For discard, update the job to discarded.
- For submit/mark applied, update it to applied with appliedAt now.
- For scan_now, return a flag requesting a cycle.
- Mark processed command records done and rewrite the JSONL file.
- Append useful activity events.

cycle.mjs:

- process commands first.
- read prefs; if autonomy is off and the cycle was not forced and there is no scan request, return a skipped result.
- append 'Hunt cycle started'.
- run scan(), then score().
- if autonomy is on and profile is not demo, call tailorAuto(8) as best-effort.
- calculate pipeline totals and write a Run record with started/ended time, boards scanned, new jobs, scored count, CV count, applied count, and parked count. Keep recent run history.
- append a success activity summary.
- if autonomy and autoSubmit are enabled and profile is not demo, spawn engine/apply.mjs detached. Pass --live only when prefs.liveApply is true; otherwise let it run dry.
- export cycle({force}) and support `node engine/cycle.mjs --force`.

mark.mjs:

Provide a small manual helper: `node engine/mark.mjs cv <jobId>`, `node engine/mark.mjs applied <jobId>`, and `node engine/mark.mjs park <jobId> <reason> "<note>"`. Use updateJob().

Run syntax checks for all engine files and tell me whether the engine can now complete one end-to-end cycle.

Useful engine commands

```
node engine/cycle.mjs --force
node engine/apply.mjs
node engine/apply.mjs --live
node engine/mark.mjs applied <jobId>
```

10. Step 8 - Build the dashboard design system

The original visual idea is a warm study dashboard rather than a generic SaaS admin panel. You can keep that style or adapt the colors to your own brand.

PASTE INTO CLAUDE CODE - PROMPT 8

Continue in web/. Create a cohesive design system for the Karmendra AI Job Hunter dashboard. Use a warm, low-saturation study aesthetic: cream paper background, soft sky blue, sage green, warm amber accents, rounded glass cards, large radii, and soft shadows. Avoid pure black and pure white.

Create web/app/fonts.ts with next/font/google:

- Zen_Maru_Gothic as the display font at weights 500, 700, 900 using variable --font-display.
- Plus_Jakarta_Sans as the UI/body font using variable --font-sans.
- Klee_One as a handwritten accent at weights 400 and 600 using variable --font-hand.

Use safe fallbacks and disable preload where appropriate for the Japanese-capable faces.

Create web/app/globals.css for Tailwind 4, tw-animate-css, and shadcn styles. Define semantic variables for paper/background, ink, muted ink, sky, sage, amber, rose, borders, shadows, glass surfaces, grade colors A-F, and day/dusk themes. Include utilities such as glass, glass-strong, paper texture, soft shadow, animated float, and subtle scene motion.

Make the app responsive and readable on a laptop screen. Keep contrast accessible. Do not build page content yet. Show me the palette and font tokens when finished.

11. Step 9 - Build the application shell

The frame gives every screen the same navigation, controls, live status, and background.

PASTE INTO CLAUDE CODE - PROMPT 9

Continue in web/. Before coding, read the installed Next.js App Router docs for this exact version. Build the reusable application shell.

Create lightweight shadcn-style components under web/components/ui using @base-ui/react + Tailwind where useful: button, card, badge, input, label, switch, slider, dialog, sheet, tabs, tooltip, select, textarea, separator, scroll-area, toaster/sonner, skeleton, avatar, progress. Add a GlassCard helper.

Create reusable visual components for grades, small status dots, relative time, empty states, and compact pills.

Build a sidebar with Karmendra AI Job Hunter branding and six routes: Dashboard, Job Scan, Applications, Resumes, Companies, Settings. Include icons from lucide-react. Add a small mascot/visual detail if you want, but keep the UI professional.

Build a top bar with agent status, `Run hunt` button, and day/dusk theme switch. `Run hunt` will later POST to /api/run-cycle.

Build a subtle background scene layer that does not interfere with readability.

Create the root layout so every page is wrapped in the shell and later in a LiveProvider. Make six routes exist even if five still have placeholder content.

When autonomy is enabled, plan for a client interval that calls /api/run-cycle every prefs.scanIntervalMin minutes. Do not implement the API route yet. Run the app and verify every navigation link routes correctly.

12. Step 10 - Build the live home dashboard

The home screen is the main control center: welcome summary, animated numbers, activity feed, top matches, and human-review tray.

PASTE INTO CLAUDE CODE - PROMPT 10

Continue in web/. Build the live client layer and the Dashboard page.

Live client layer:

- web/lib/use-live.ts: useLive(initialSnapshot) starts from the server snapshot, opens EventSource('/api/stream'), and replaces state on `snapshot` and `update` events. Reconnect sensibly after disconnects.
- Add useNow(intervalMs around 30000) for relative timestamps.
- Add sendCommand(type, payload), savePrefs(patch), and runCycle() helpers using fetch POSTs.
- web/components/live-provider.tsx: LiveProvider + useSnapshot() context hook.

Dashboard components:

- Hero/welcome card summarizing roles scanned, companies/boards, CVs prepared, sent applications, and parked items using only snapshot data.
- Three large animated stats: roles swept, applications sent, interviews.
- Live Active Hunt/Activity Feed: newest first; show a phase icon, level/grade styling, message, company where relevant, and relative time. Animate new rows and highlight the newest.
- Top Matches: strongest jobs sorted by score with grade, role, company, location, and match percentage.
- Needs You tray: parked jobs with reason/note, `Open posting`, `Mark applied`, and `Not for me` actions that queue commands.
- A small current-run/agent status card.

Read the initial snapshot on the server where appropriate, then let LiveProvider take over on the client. Make the page responsive and laptop-friendly. Use the design system from the previous steps.

Do not hard-code demo numbers. Everything displayed should come from Snapshot. Verify that the dashboard renders even when jobs/activity are empty.

13. Step 11 - Build the remaining five screens

These screens make the tool usable beyond the reel-style home page.

PASTE INTO CLAUDE CODE - PROMPT 11

Continue in web/. Build the remaining five pages as client screens that read from useSnapshot(). Add a compact PageHeader component for title, subtitle, and optional action.

1. Job Scan - /scan

Title: Job Scan. Show boards swept, roles found, and new this cycle. Add a Scan now button that queues scan_now. Show enabled sources with ATS label and role count, plus the newest discovered jobs.

2. Applications - /applications

Build a horizontal kanban. Use columns for discovery/scored, CV ready, Needs You, Applied, Interview, Offer/Closed (group statuses cleanly). Each column shows count and sorted JobCards. Empty columns say `nothing here yet`.

3. JobCard + JobDetail Sheet

Create web/components/job-card.tsx. The compact card shows grade, title, company/location, match percentage, archetype, and any park reason. Clicking opens a right Sheet with legitimacy, score, reasons, gaps, park note, posting link, tailored-CV link, Mark applied, and Not for me. Actions queue commands.

4. Resumes - /resumes

Left list: Base CV plus each tailored CV. Right panel: selected Markdown rendered with react-markdown, grade/meta strip, role/company/match, and an ATS-tuned badge. Load content from /api/cv?id=. If ?job=<id> is present, select that CV on load.

5. Companies - /companies

Show enabled count and total. Add a company dialog with name, ATS (Greenhouse, Lever, Ashby), and board token. POST changes to /api/companies. Company cards show initial, ATS, enabled switch, role count, and careers link.

6. Settings - /settings

Autonomy controls: hunt on its own, auto-submit clean forms, and live apply. Live apply must display a strong warning; off = dry run. Add minimum auto-submit score slider 3-5, daily cap 1-30, per-company cap 1-5, and scan interval choices 15m/30m/1h/2h.

Profile section: full name, email, location, headline, target roles, compensation floor/target. Saving sets onboarded true and isDemo false.

Truthful Answers section: work authorization, sponsorship, notice period, relocation, salary expectation, start date, referral source, criminal disclosure, and EEO fields defaulting to `Decline to self-identify`. Clearly state that unknown questions are parked.

Base CV section: large Markdown textarea saved as id `base`. Never overwrite the original as part of tailoring.

Dataset section: optional Real vs Demo control for filming/testing.

Run lint/type checks and verify all six pages navigate without errors.

14. Step 12 - Add live streaming and API routes

The dashboard should update when the engine writes to disk, without constantly refreshing the whole page.

PASTE INTO CLAUDE CODE - PROMPT 12

Continue in web/. Read the route-handler documentation installed with this version of Next.js. Build all local API routes. For each route use the Node.js runtime and force dynamic behavior where required.

1. /api/stream GET - Server-Sent Events. Send a full snapshot immediately as event `snapshot`. Every 1 second compare a stat signature (mtime + size) for shared files; when it changes, send event `update` with a new snapshot. Send an SSE comment ping every 15 seconds. Clean up timers when the request aborts. Use headers for text/event-stream, no-cache, keep-alive, and disable proxy buffering.
2. /api/run-cycle POST - start a real engine cycle from the dashboard. Prevent accidental overlapping runs with a time-based lock file under data/.cycle-lock; if touched within about 6 minutes, return busy. Otherwise update the lock and spawn `node engine/cycle.mjs --force` detached from the repository root. Return a JSON started/busy result.
3. /api/command POST - validate command type against the allowed list, then append it with queueCommand().
4. /api/prefs POST - merge a patch into current prefs and save it.
5. /api/profile POST - validate basic fields and save profile.
6. /api/companies POST - support adding a company and toggling an existing board enabled/disabled.
7. /api/cv GET ?id= - return Markdown for base or a job-specific CV. POST {id, markdown} writes it safely.
8. /api/state GET - return the complete current Snapshot as JSON.
9. /api/snapshot POST - if you keep the demo/real switch, seed demo data or reset to a real empty dataset without deleting the user's base CV/profile unless explicitly requested.

Create .claude/commands/hunt-cycle.md. The slash command should instruct the active Claude Code session to run one engine cycle, inspect newly scored A/B jobs against the actual base CV and available job details, improve only evidence-backed reasons/gaps/score, generate truthful tailored CVs for strong fits, and run the apply preparation/self-review gate. It must never submit when profile.isDemo is true, and it must stop after one cycle.

Wire the top Run hunt button to /api/run-cycle and confirm the browser updates within roughly one second after a data file changes. Run lint and build checks.

Manual Claude Code command after this step

```
/hunt-cycle
```

15. Step 13 - Seed the dashboard and run everything

The seeder makes the interface immediately useful for testing. It should create a demo profile that explicitly blocks real submission.

PASTE INTO CLAUDE CODE - PROMPT 13

Continue Karmendra Job Hunter. Create engine/seed.mjs and then verify the full application.

Seeder behavior:

- Fetch current public postings from a small set of known Greenhouse and Lever boards. Board tokens can change, so failures must be skipped gracefully rather than crashing the entire seed.
- Greenhouse examples to try: anthropic, vercel, airtable, temporal, arizeai, gleanwork, speechmatics, planetscale, hightouch, runwayml, wayve, stabilityai.
- Lever examples to try: mistral, spotify.
- Keep job titles related to AI, ML, LLM, agents, platform, solutions, automation, product, engineering, data, and similar target areas. Exclude intern/junior/mobile/embedded examples when they are clearly outside the demo target.
- Score demo jobs with a simple deterministic heuristic and spread them across the pipeline so the dashboard has examples of interview, applied, needs_you, cv_ready, and scored states.
- Write a DEMO profile with placeholder identity, Remote / Worldwide location, target roles such as AI Engineer, Solutions Architect, and Automation Engineer, currency USD, workAuthorized true, requiresSponsorship false, and isDemo true.
- Write sensible prefs with liveApply false.
- Write companies, jobs, a run record, activity events, and a simple Markdown base CV with Summary and Experience headings.
- Never submit a real application from seed mode.

After creating the seeder, run these in order and fix any errors before continuing:

1. npm install
2. npx playwright install
3. npm --prefix web install
4. node engine/seed.mjs
5. npm --prefix web run dev

Tell me what should appear at <http://localhost:4319>. Confirm that the demo profile blocks real submissions and live apply is still off.

Run commands - exact order

```
npm install
npx playwright install
npm --prefix web install
node engine/seed.mjs
npm --prefix web run dev
```

Open <http://localhost:4319>. You should now see populated jobs, scores, the activity feed, pipeline states, and a Needs You queue.

16. Switch from demo to your real profile

1. Open Settings in the dashboard.
2. Paste your actual CV into Base CV. Keep it factual and complete.
3. Enter your real name, email, location, headline, and target roles.
4. Fill work authorization, sponsorship, relocation, notice period, salary expectation, start date, referral source, criminal disclosure, and any EEO answers you choose to provide.
5. Save the profile. This should set isDemo to false.
6. Keep Live Apply OFF.
7. Turn Autonomy ON only after the profile, CV, and company list are correct.
8. Choose a minimum score and conservative daily/per-company caps.
9. Run one /hunt-cycle and inspect the discovered jobs, tailored CVs, and dry-run screenshots.
10. Only after you trust the field mapping should you consider enabling Live Apply.

Run one cycle from Claude Code

```
/hunt-cycle
```

Or run the raw engine without the Claude review layer:

```
node engine/cycle.mjs --force
```

Let Claude Code run the hunt on a timer

If your Claude Code environment supports the loop command, run:

```
/loop 30m /hunt-cycle
```

KEEP THE SESSION OPEN This workflow is local. The dashboard, Node engine, Playwright, and Claude Code session need to be available on the computer for scheduled local cycles to work.

17. How to add companies

For Greenhouse and Lever, the scanner uses the board token rather than scraping a careers page.

Greenhouse

A Greenhouse company feed uses:

```
https://boards-api.greenhouse.io/v1/boards/<token>/jobs
```

Example company record shape:

```
{
  "name": "Example Co",
  "ats": "greenhouse",
  "token": "example-token",
  "careersUrl": "https://example.com/careers",
  "enabled": true
}
```

Lever

```
https://api.lever.co/v0/postings/<token>?mode=json
```

If a company changes ATS or its token stops working, disable that source instead of repeatedly retrying a broken endpoint.

Ashby / Workable / Workday

The base build leaves some providers incomplete on purpose. Only add a provider when you have verified a legitimate public feed or a permitted integration. Do not turn the project into an indiscriminate scraper.

18. Dry run vs live apply

Treat dry run as the normal operating mode until the entire profile and form mapping have been reviewed.

Mode	What happens	Use it when
Demo	Shows realistic jobs/pipeline. Real submit is blocked by profile.isDemo.	UI testing and filming.
Dry run	Browser fills what it can, captures evidence, but does not press final submit.	Testing your real profile safely.
Live	Clean forms can be submitted when every safety check passes.	Only after manual review of multiple dry runs.

A job should be parked when

- A CAPTCHA is present.
- Email verification or account creation is required.
- A required field is not mapped to a truthful profile value.
- A legal/identity question is ambiguous.
- The page requires a login or unsupported workflow.
- The final submit result cannot be confidently confirmed.
- A validation error remains after filling the form.

19. Validation checklist before turning on live apply

- The base CV contains all facts you want the tailor to use.
- At least five tailored CVs were manually reviewed and contain no invented claims.
- Work authorization and sponsorship settings are correct.
- Salary expectation and notice period are intentional.
- Daily cap is conservative.
- Per-company cap prevents duplicate over-application.
- Score threshold is high enough that you would personally apply to most A/B roles.
- Multiple dry runs correctly parked unknown or risky questions.
- CAPTCHAs are always parked, never bypassed.
- The company sources you enabled are legitimate public job feeds or otherwise permitted.
- You have checked the relevant career site's terms and are comfortable with your automation behavior.

Engineering checks

```
npm --prefix web run lint
npm --prefix web run build
node engine/cycle.mjs --force
```

If the build fails, paste the complete error into the same Claude Code session and ask it to fix the smallest root cause without redesigning working parts.

20. Troubleshooting

Dashboard does not open on port 4319

Another process may hold the port or the dev server may not be running. Stop the old process and rerun:

```
npm --prefix web run dev
```

No jobs appear

- Confirm companies.json has enabled sources.
- Confirm Greenhouse/Lever tokens are correct.
- Run node engine/scan.mjs directly and inspect the output.
- Check data/log/activity.jsonl for scan results.
- Temporarily broaden target roles/keywords if your filters are too narrow.

Playwright cannot open a browser

```
npx playwright install
```

Almost every job is parked

That can be correct. Modern application flows frequently have login walls, CAPTCHAs, verification steps, or custom questions. A conservative system should park rather than invent.

Nothing submits

- Confirm profile.isDemo is false.
- Confirm autonomy is on.
- Confirm auto-submit is on.
- Confirm liveApply is on only when you intend real submission.
- Confirm the score meets the minimum threshold.
- Confirm daily and per-company caps have not been reached.

Tailored CV is generic

Improve data/cv/base.md. The tailoring stage can only emphasize what you provide; it should not invent missing achievements.

Dashboard is not updating live

- Check /api/state returns JSON.
- Check /api/stream stays open as an EventSource.
- Confirm shared data paths resolve to the same data/ folder.
- Check that files actually change after a cycle.
- Restart the Next.js dev server after route changes.

21. Command cheat sheet

Keep this page nearby while you build.

Create and enter the project

```
mkdir karmendra-job-hunter  
cd karmendra-job-hunter
```

Start Claude Code

```
claude --dangerously-skip-permissions --model best  
/effort
```

Install dependencies

```
npm install  
npx playwright install  
npm --prefix web install
```

Seed and start the dashboard

```
node engine/seed.mjs  
npm --prefix web run dev
```

Run a hunt

```
/hunt-cycle  
node engine/cycle.mjs --force
```

Run a repeating Claude Code cycle

```
/loop 30m /hunt-cycle
```

Application engine

```
node engine/apply.mjs          # dry run  
node engine/apply.mjs --live # real submit when allowed
```

Quality checks

```
npm --prefix web run lint  
npm --prefix web run build
```

22. Final build checklist

1. Engine and dashboard folders exist.
2. Next.js opens at localhost:4319.
3. Profile, preferences, companies, jobs, runs, activity, commands, and CV files share the same data/directory.
4. Greenhouse and Lever scanners work for at least a few enabled companies.
5. New jobs are deduplicated.
6. Scorer gives A-F grade, match percentage, reasons, and gaps.
7. A/B jobs can receive a truthful tailored Markdown CV and PDF.
8. Apply engine defaults to dry-run.
9. CAPTCHAs and unknown required questions are parked.
10. Dashboard shows live feed, top matches, pipeline, Needs You, companies, resumes, and settings.
11. Run hunt triggers an engine cycle.
12. /hunt-cycle exists in .claude/commands/.
13. Demo mode blocks real submission.
14. Your real profile has been reviewed before any live apply toggle is enabled.

YOU ARE DONE WHEN You can start the dashboard, run one dry hunt, watch new jobs arrive, inspect their scores, open a tailored CV, and see difficult applications move into Needs You without the system fabricating an answer.

Karmendra AI - build useful AI systems, understand what they are doing, and keep humans in control where judgment matters.